



CS251: Software Engineering

Assignment -1

Dr M. El-Ramly

Section(19,20)

Language: (JAVA)

Team Members Details :

Name	ID
Kevin Remon George	20240431
Salma Mohamed Mostafa	20221074
Mohamed Ibrahim Elshafey	20242426
Hamza Mohamed Ahmed	20242422

Task 1:

Name	Hours Spent	Learning Resources	Project Name
Salma Mohammed	10	1.El dosoki 2.W3school 3.Et3alem Bbasata	Simple bank System
Kevin Remon	16	Adel nessem google search	To Do List
Mohamed Ibrahim	9	1- ElDosoki 2- Adel Nasseem	Cinema Booking System
Hamza Mohamed	12	1- Adel Nasim 2- Mohamed El-Desouki	Library System

Screenshots of Tutorials & Learning Materials

(Salma Mohammed)

اليوم

كورس لغة جافا من الصفر | Java Course
لتعلم ببساطة • 227 ألف مشاهدة
مل على التي 3 كورس لغة جافا مع الكتب والتمارين 1 <https://learnsimply.com/courses/java-course-level1> كورس الترمجة للكتابة...



ترجمة 2 - 16 - Final Methods and Protected Members in Java
محدث التسوي (Mohamed El Desouki) • 127 ألف مشاهدة
in this video you will learn how to declare Final methods and protected members in java by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة...
Java... - السعودية - الخرج



24 - Java Programming for Beginners - Two Dimensional Arrays
محدث التسوي (Mohamed El Desouki) • 154 ألف مشاهدة
by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة الأمير سلطان بن عبد العزيز...
Java... - السعودية - الخرج



00 - Java Programming for Beginners - Introduction
محدث التسوي (Mohamed El Desouki) • 1.1 مليون مشاهدة
by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة الأمير سلطان بن عبد العزيز...
Java... - السعودية - الخرج



01 - | Java - Data Structures For Beginners | - | Why To Learn Data Structures?
محدث التسوي (Mohamed El Desouki) • 41 ألف مشاهدة
شرح Dr. Mohamed El Desouki mohamed_eldesouki@hotmail.com Data Structures in Java For Beginners
...تركيب البيانات - شرح هياكل البيانات مرفق الكود



01 - Java Programming for Beginners - What is programming ?
محدث التسوي (Mohamed El Desouki) • 567 ألف مشاهدة
by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة الأمير سلطان بن عبد العزيز...
Java... - السعودية - الخرج



أمس

01 - Java Programming for Beginners - What is programming ?
محدث التسوي (Mohamed El Desouki) • 567 ألف مشاهدة
by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة الأمير سلطان بن عبد العزيز...
Java... - السعودية - الخرج



00 - Java Programming for Beginners - Introduction
محدث التسوي (Mohamed El Desouki) • 1.1 مليون مشاهدة
by : Mohamed El Desouki mohamed_eldesouki@hotmail.com Tel :00966 553450836 جامعة الأمير سلطان بن عبد العزيز...
Java... - السعودية - الخرج



W3schools

Tutorials References Exercises Certificates Search...

HTML CSS JAVASCRIPT SQL PYTHON **JAVA** PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

Java Syntax
Java Output
Java Comments
Java Variables
Java Data Types
Java Type Casting
Java Operators
Java Strings
Java Math
Java Booleans
Java If...Else
if
else
else if
Short Hand If...Else
Nested If
Logical Operators
Real-Life Examples
Code Challenge
Java Switch
Java While Loop
Java For Loop
Java Break/Continue

Java If ... Else

< Previous

Next >

Java Conditions and If Statements

Conditions and if statements let you control the flow of your program - deciding which code runs, and which code is skipped.

Think of it like real life: *If it rains, take an umbrella. Otherwise, do nothing.*

Every **if** statement needs a condition that results in **true** or **false**.

This means **if** statements work hand-in-hand with **boolean** values:

Example

```
boolean isRaining = true;

if (isRaining) {
    System.out.println("Bring an umbrella!");
}
```

Get your own Java Server

Projects: *(Simple bank System)*

Description:

"Simple Bank System" is a Java console app . Users can Deposit, Withdraw, Check Balance , **and** Exit

Overview

This Java program is a **Bank Account Management system** designed to simulate basic banking operations. It uses **Object-Oriented Programming (OOP)** to manage user data and ensure secure transactions.

Key Components

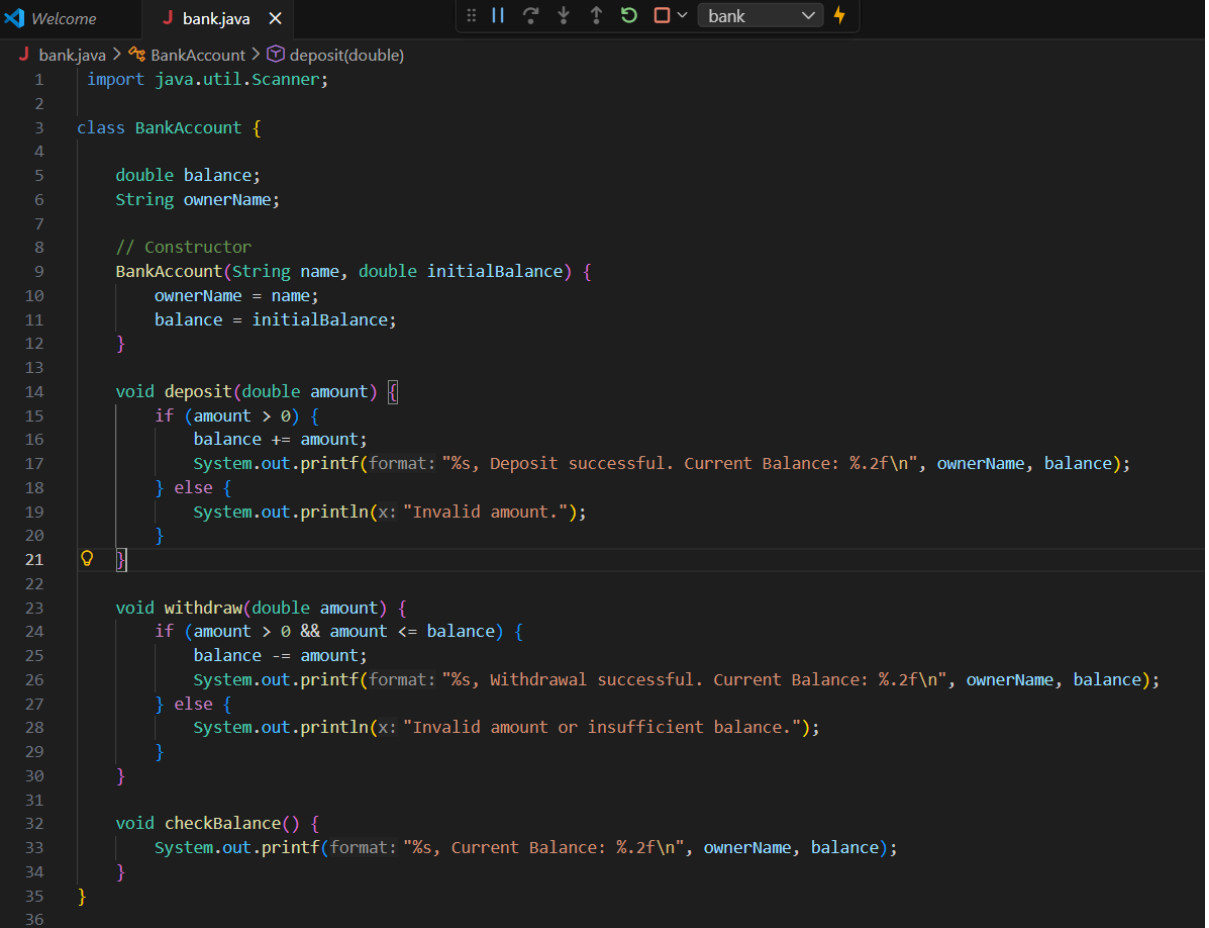
- **BankAccount Class:** Stores the account holder's name and balance. It handles the logic for **depositing**, **withdrawing**, and **viewing** funds.
- **Safety Features:** The code includes validation to prevent invalid actions, such as withdrawing more money than available or depositing negative amounts.
- **Interactive Menu:** A **do-while** loop and a **switch** statement allow the user to perform multiple transactions in one session until they choose to exit.

Summary of Operations

Action	Description
Deposit	Adds a positive amount to the current balance.
Withdraw	Deducts funds only if the balance is sufficient.
Check Balance	Displays the current balance and owner's name.
Exit	the program closes and says "Goodbye".

In short: This code is an excellent example of object-oriented programming (OOP) because it treats the bank account as an "object" with attributes (name and balance) and actions (withdrawal and deposit). The program manages this relationship in a way that prevents errors (such as withdrawing more than the balance).

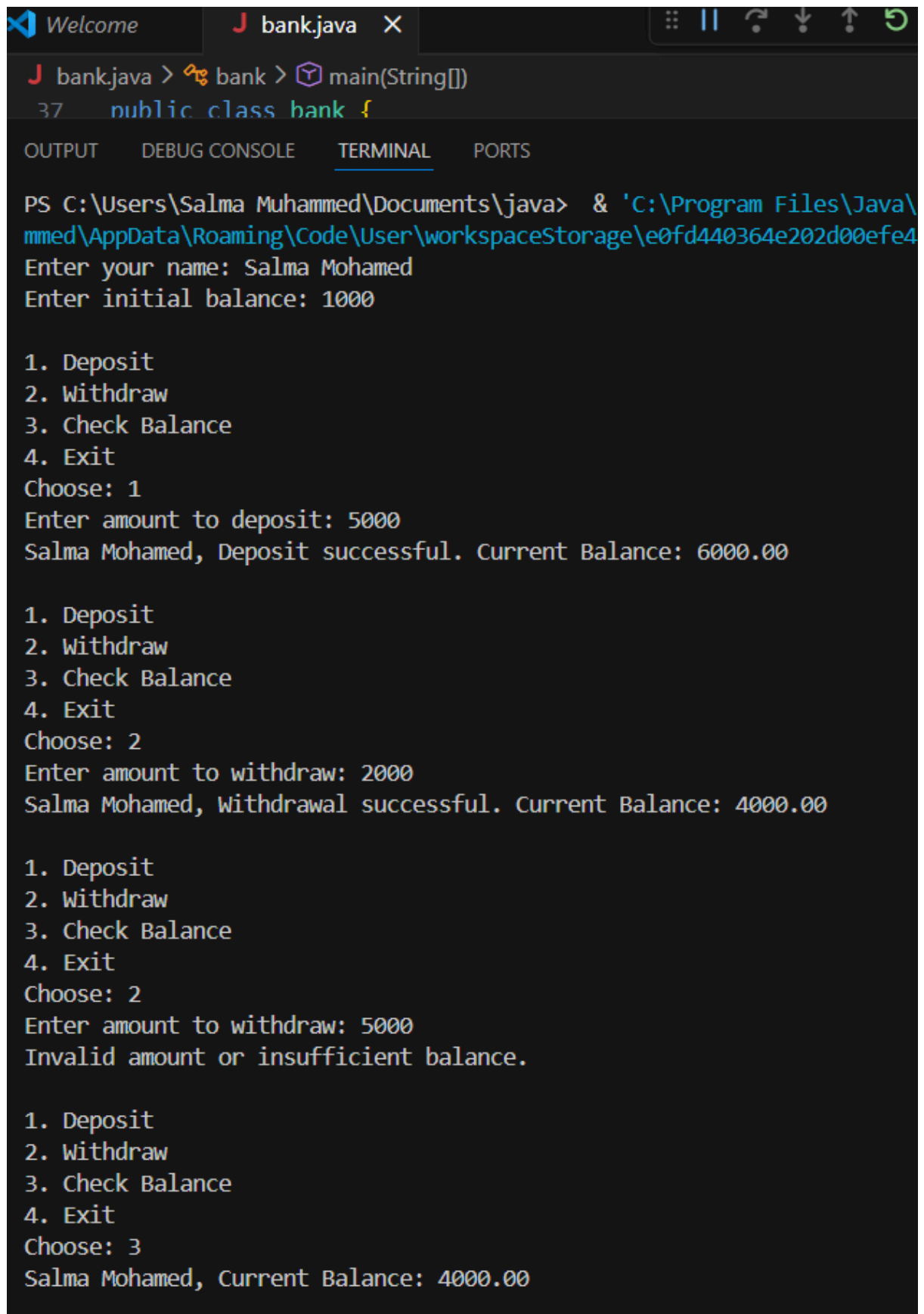
Screenshots to the App:

A screenshot of a Java IDE window titled 'bank.java'. The code defines a 'BankAccount' class with attributes 'balance' (double) and 'ownerName' (String). It includes a constructor 'BankAccount(String name, double initialBalance)' and three methods: 'deposit(double amount)', 'withdraw(double amount)', and 'checkBalance()'. The 'deposit' method checks if the amount is positive and updates the balance accordingly. The 'withdraw' method checks if the amount is positive and does not exceed the current balance. The 'checkBalance' method prints the current balance. The IDE interface shows a dark theme with line numbers on the left and a toolbar at the top.

```
1  import java.util.Scanner;
2
3  class BankAccount {
4
5      double balance;
6      String ownerName;
7
8      // Constructor
9      BankAccount(String name, double initialBalance) {
10         ownerName = name;
11         balance = initialBalance;
12     }
13
14     void deposit(double amount) {
15         if (amount > 0) {
16             balance += amount;
17             System.out.printf(format: "%s, Deposit successful. Current Balance: %.2f\n", ownerName, balance);
18         } else {
19             System.out.println(x: "Invalid amount.");
20         }
21     }
22
23     void withdraw(double amount) {
24         if (amount > 0 && amount <= balance) {
25             balance -= amount;
26             System.out.printf(format: "%s, Withdrawal successful. Current Balance: %.2f\n", ownerName, balance);
27         } else {
28             System.out.println(x: "Invalid amount or insufficient balance.");
29         }
30     }
31
32     void checkBalance() {
33         System.out.printf(format: "%s, Current Balance: %.2f\n", ownerName, balance);
34     }
35 }
36
```

```
Welcome J bank.java X
J bank.java > bank > main(String[])
37 public class bank {
39     public static void main(String[] args) {
40
41         Scanner input = new Scanner(System.in);
42
43         System.out.print(s: "Enter your name: ");
44         String name = input.nextLine();
45
46         System.out.print(s: "Enter initial balance: ");
47         double initialBalance = input.nextDouble();
48
49         BankAccount account = new BankAccount(name, initialBalance);
50
51         int choice;
52
53         do {
54             System.out.println(x: "\n1. Deposit");
55             System.out.println(x: "2. Withdraw ");
56             System.out.println(x: "3. Check Balance ");
57             System.out.println(x: "4. Exit ");
58             System.out.print(s: "Choose: ");
59
60             choice = input.nextInt();
61
62             switch (choice) {
63                 case 1:
64                     System.out.print(s: "Enter amount to deposit: ");
65                     account.deposit(input.nextDouble());
66                     break;
67
68                 case 2:
69                     System.out.print(s: "Enter amount to withdraw: ");
70                     account.withdraw(input.nextDouble());
71                     break;
72
73                 case 3:
74                     account.checkBalance();
```

Simple bank system console App:



The screenshot shows a Java IDE with a tab for 'bank.java'. The code editor displays the following code:

```
37 public class bank {
```

Below the code editor, the 'TERMINAL' tab is active, showing the execution of the program. The command prompt shows the user running the program from the directory 'C:\Users\Salma Muhammed\Documents\java'. The program prompts the user to enter their name and initial balance. The user enters 'Salma Mohamed' and '1000'. The program then displays a menu with four options: 1. Deposit, 2. Withdraw, 3. Check Balance, and 4. Exit. The user chooses option 1 (Deposit) and enters an amount of 5000. The program displays the current balance as 6000.00. The user then chooses option 2 (Withdraw) and enters an amount of 2000. The program displays the current balance as 4000.00. The user then chooses option 2 (Withdraw) again and enters an amount of 5000. The program displays an error message: 'Invalid amount or insufficient balance.' The user then chooses option 3 (Check Balance) and the program displays the current balance as 4000.00.

```
PS C:\Users\Salma Muhammed\Documents\java> & 'C:\Program Files\Java\
mmed\AppData\Roaming\Code\User\workspaceStorage\e0fd440364e202d00efe4
Enter your name: Salma Mohamed
Enter initial balance: 1000

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose: 1
Enter amount to deposit: 5000
Salma Mohamed, Deposit successful. Current Balance: 6000.00

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose: 2
Enter amount to withdraw: 2000
Salma Mohamed, Withdrawal successful. Current Balance: 4000.00

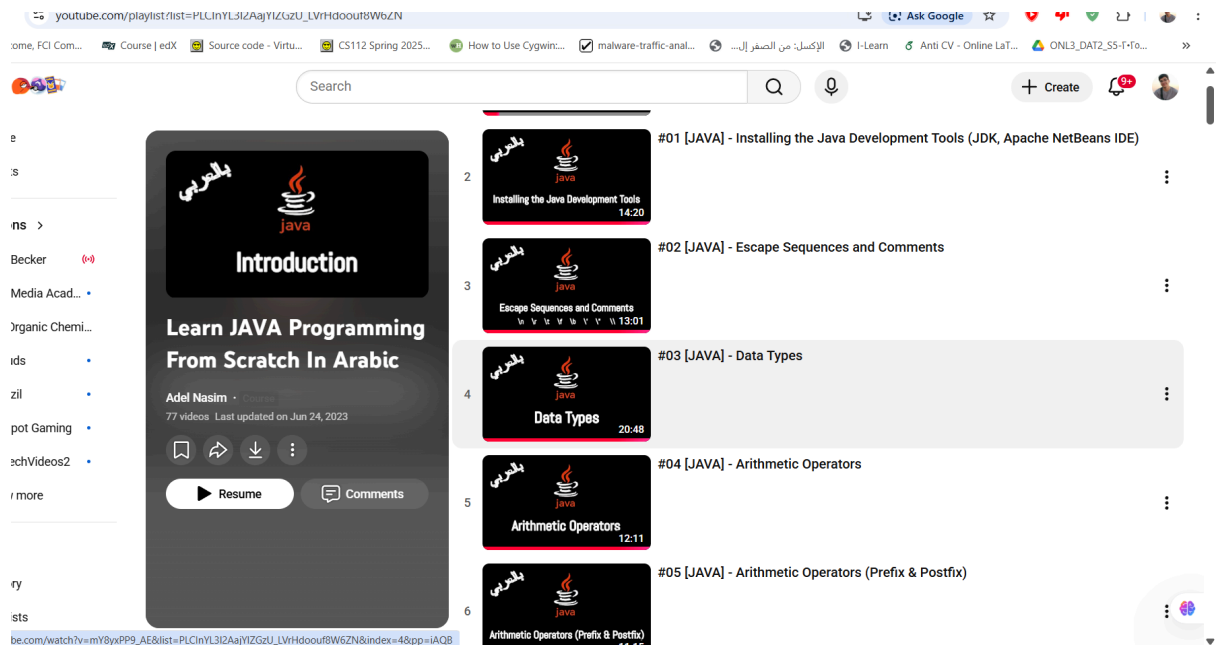
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose: 2
Enter amount to withdraw: 5000
Invalid amount or insufficient balance.

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose: 3
Salma Mohamed, Current Balance: 4000.00
```


LINK VIDEO:[ClickHere](#)

Screenshots of Tutorials & Learning Materials

(Kevin Remon)



Project: To Do List

Description:

"To do List" is a Java console app . Users can Add tasks, view tasks, Delete tasks , Mark tasks done **and** Exit

Overview

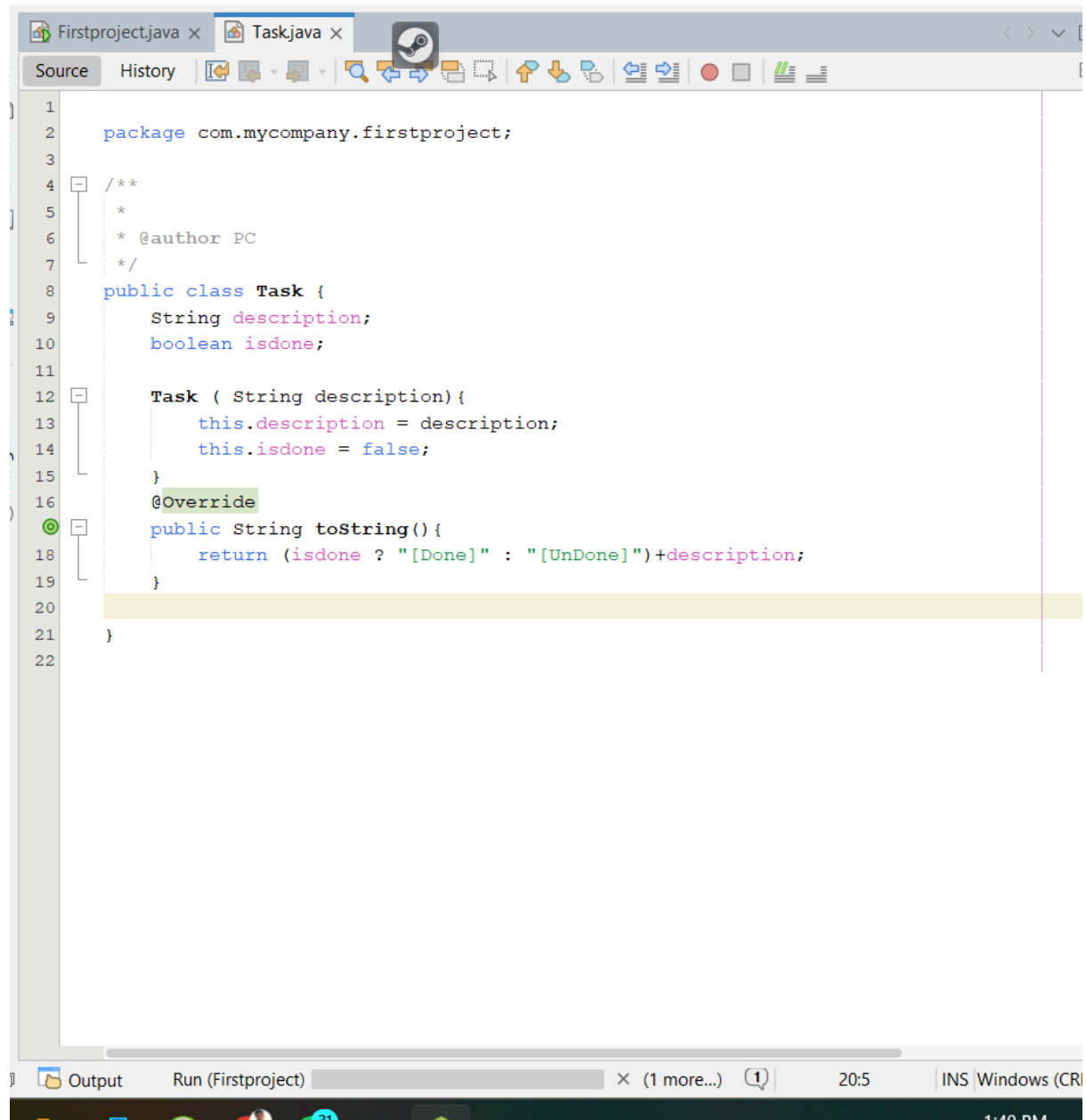
This Java program is a **simple List system** designed to simulate a real to do list sheet . It uses **Object-Oriented Programming** to manage user data.

Key Components

- **Task Class:** represents a to do item description and completion status, it tracks whether the task is done or not.
- **Safety Features:** The code includes validation to prevent invalid actions, such as inputting wrong task numbers or if there are no task to mark done
- **Interactive Menu:** a **switch** statement allow the user to perform multiple task insertions in one session until they choose to exit.

Summary of Operations

Action	Description
Add task	Adds a Task to the list
View Task	Shows the tasks inserted and their status
delete task	removes a task from list
Mark as done	changes the task status to done



```
1 package com.mycompany.firstproject;
2
3
4 /**
5  *
6  * @author PC
7  */
8 public class Task {
9     String description;
10    boolean isdone;
11
12    Task ( String description){
13        this.description = description;
14        this.isdone = false;
15    }
16    @Override
17    public String toString(){
18        return (isdone ? "[Done]" : "[UnDone]")+description;
19    }
20
21 }
22
```

Output Run (Firstproject) × (1 more...) 20:5 INS Windows (CR)

1:40 PM

```
3
4
5 do{
6     System.out.println("--- To Do List ---");
7     System.out.println ( " 1- Add Task");
8     System.out.println ( " 2- View Task");
9     System.out.println ( " 3- Mark Task as Done");
10    System.out.println( " 4- Delete Task");
11    System.out.println( " 0- Exit Program");
12    System.out.println ( "Enter your Choice: ");
13    choice = scanner.nextInt();
14    scanner.nextLine();
15
16    switch(choice){
17        case 1:
18            addTask(scanner);
19            break;
20
21        case 2:
22            viewTasks();
23            break;
24        case 3:
25            MarkTask(scanner);
26            break;
27        case 4:
28            deleteTask(scanner);
29            break;
30        case 0:
31            System.out.println ( " GoodBye");
32            break;
33
34        default:
35            System.out.println("Invalid choice");
36    }
37
38    } while(choice != 0);
39 }
```

Output Run (First project) X (1 more) (1) 42:28 INS | Univ (LE)

```
Output - Run (Firstproject)

Enter Task number to mark done:
3
Task Marked done successfully
--- To Do List ---
1- Add Task
2- View Task
3- Mark Task as Done
4- Delete Task
0- Exit Program
Enter your Choice:
2
1.[Done]I
2.[Done]Love
3.[Done]Dr. Ramli
--- To Do List ---
1- Add Task
2- View Task
3- Mark Task as Done
4- Delete Task
0- Exit Program
Enter your Choice:
|
```

Video link:

https://drive.google.com/file/d/1CDD6QttXvJl4aZL6n5EXYOFuhSqwCXyZ/view?usp=drive_link

Screenshots of Tutorials & Learning Materials

(Mohamed Ibrahim)



Java Programming For Beginners - Course 1- بالعربي
(Mohamed El Desouki) محمد الدسوقي · Course
56 videos · Last updated on Jan 31, 2022

Resume Comments

سلسلة تعليمية لتعليم الجافا للمبتدئين

8  **Programming For Begi** 17:51

9  **Programming For Begi** 12:51

10  **Programming For Begi** 21:46

11  **Programming For Begi** 19:23

12  **Programming For Begi** 17:28

13 

08 - Java Programming for Beginners -Arithmetic Operators - Part 2

09 - Java Programming for Beginners - IF statement - Part 1

10 - Java Programming for Beginners - IF statement - Part 2

11 - Java Programming for Beginners - Switch statement - Part 1

12 - Java Programming for Beginners - Switch statement - Part 2



Introduction

Learn JAVA Programming From Scratch In Arabic
Adel Nasim · Course
77 videos · Last updated on Jun 24, 2023

Resume Comments

6  **Arithmetic Operators (Prefix & Postfix)** 11:15

7  **Java User Input (Scanner Class), Printf methods** 18:33

8  **Selection Statement (if - if else - if else if)** 16:55

9  **Selection Statement (Conditional * Ternary * Operator), Instanceof Method** 13:41

10  **Switch Statements** 18:56

#05 [JAVA] - Arithmetic Operators (Prefix & Postfix)

#06 [JAVA] - Java User Input (Scanner), Printf methods

#07 [JAVA] - Selection Statement (if - if else - if else if)

#08 [JAVA] - Selection Statement (Exercises, Practice, Ternary Operator),...

#09 [JAVA] - Selection Statement (Switch Statements)

Project: Cinema Booking System

Description:

The Cinema Booking System is a Java console program that allows a cinema cashier to manage movie screenings, view seating arrangements, book tickets, and track total revenue. The program supports multiple halls of fixed sizes and starts with empty movie slots that the cashier can populate.

Key Components

- **Movie Class** represents a movie screening in a hall, storing its **title**, **ticket price**, and seating chart. Provides methods to display seats and **update movies**.
- **Seating System** A 2D character array for each movie representing available ('O') and booked ('X') seats.
- **Revenue Tracking** A variable to store and update total **revenue** from booked tickets.
- **Main Application (App Class)** Contains the menu-driven interface and handles user input, booking logic, and revenue calculation.

Summary of Operations

Action	Description
List Movies	Displays all halls with current movies and their ticket prices.
Add Or Change Movie	Allows the cashier to assign or update a movie in any hall
Book Ticket	Lets the user select a hall, view the seating chart, and reserve a seat.
Show Revenue	Displays the total revenue collected so far.
Exit	The program closes and says, "Goodbye."

Screenshots of the App:

```
App.java
src > App.java > Movie > updateMovie(Movie[], Scanner)
1 import java.util.Scanner;
2
3 public class App {
4     public static void main(String[] args) throws Exception {
5         System.out.println(x: "Cinema Booking System");
6         Scanner input = new Scanner(System.in);
7         double revenue = 0;
8
9         Movie[] movies = new Movie[3];
10
11         while (true) {
12             System.out.println(x: "1. List The Movies");
13             System.out.println(x: "2. Add/Change Movie");
14             System.out.println(x: "3. Book Ticket");
15             System.out.println(x: "4. Show Revenue");
16             System.out.println(x: "5. Exit");
17
18             int x = input.nextInt();
19             if (x == 1) {
20                 for (int i = 0; i < movies.length; i++) {
21                     if (movies[i] == null) {
22                         System.out.println((i + 1) + ". [EMPTY] No movie assigned");
23                     } else {
24                         System.out.println((i + 1) + ". " + movies[i].title + " - Price: " + movies[i].price);
25                     }
26                 }
27                 System.out.println();
28             } else if (x == 2) {
29                 Movie.updateMovie(movies, input);
30             } else if (x == 3) {
31
32                 System.out.print(s: "Enter movie number to book: ");
33                 int movieNumber = input.nextInt() - 1;
34

```

```

35         if (movies[movieNumber] == null) {
36             System.out.println(x: "This hall has no movie assigned!");
37             continue;
38         }
39
40         movies[movieNumber].showSeats();
41
42         System.out.print("Enter row (1-" + movies[movieNumber].seats.length + "): ");
43         int row = input.nextInt() - 1;
44
45         System.out.print("Enter column (1-" + movies[movieNumber].seats[0].length + "): ");
46         int col = input.nextInt() - 1;
47
48         if (movies[movieNumber].seats[row][col] == 'O') {
49             movies[movieNumber].seats[row][col] = 'X';
50             revenue += movies[movieNumber].price;
51             System.out.println("Seat booked successfully! Ticket price: " + movies[movieNumber].price);
52         } else {
53             System.out.println(x: "Sorry, that seat is already booked!");
54         }
55     } else if (x == 4) {
56         System.out.println("Total revenue so far: " + revenue + " EGP");
57     }
58
59     else if (x == 5) {
60         System.out.print(s: "Good Bye!!");
61         break;
62     }
63 }

```

```

J Appjava
src > J Appjava > Movie > updateMovie(Movie[], Scanner)

71 class Movie {
72     String title;
73     int price;
74     char[][] seats;
75
76     Movie(String title, int price, int rows, int cols) {
77         this.title = title;
78         this.price = price;
79         seats = new char[rows][cols];
80
81         for (int i = 0; i < rows; i++) {
82             for (int j = 0; j < cols; j++) {
83                 seats[i][j] = '0';
84             }
85         }
86     }
87
88     void showSeats() {
89         System.out.println("Seating for " + title + ":");
90         for (int r = 0; r < seats.length; r++) {
91             for (int c = 0; c < seats[r].length; c++) {
92                 System.out.print(seats[r][c] + " ");
93             }
94             System.out.println();
95         }
96     }
97
98     static void updateMovie(Movie[] movies, Scanner input) {
99         System.out.println(x: " Hall 1 ----> Small\n Hall 2 ----> Medium\n Hall 3 ----> Large");
100        System.out.print(s: "Enter hall number to update (1-3): ");
101        int hall = input.nextInt();
102        input.nextLine();
103
104        if (hall <= 0 || hall > movies.length) {
105            System.out.println(x: "Invalid hall!");

```

```

Cinema Booking System
1. List The Movies
2. Add/Change Movie
3. Book Ticket
4. Show Revenue
5. Exit
2
Hall 1 ----> Small
Hall 2 ----> Medium
Hall 3 ----> Large
Enter hall number to update (1-3): 2
Enter new movie title: Cinema Paradiso
Enter new ticket price: 80
Movie added/updated successfully!
1. List The Movies
2. Add/Change Movie
3. Book Ticket
4. Show Revenue
5. Exit
1
1. [EMPTY] No movie assigned
2. Cinema Paradiso - Price: 80
3. [EMPTY] No movie assigned

1. List The Movies
2. Add/Change Movie
3. Book Ticket
4. Show Revenue
5. Exit
3
Enter movie number to book: 2
Seating for Cinema Paradiso :
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
Enter row (1-6): 3
Enter column (1-8): 3
Seat booked successfully! Ticket price: 80
1. List The Movies
2. Add/Change Movie
3. Book Ticket
4. Show Revenue
5. Exit

```

Video link:

<https://drive.google.com/file/d/1Lqxafy0ZP-46YVtQWHWrW6LaZkpaF6X/view?usp=sharing>

Repo link:

<https://github.com/Citizen-Zodiac/Cinema-Booking-System>

Screenshots of Tutorials & Learning Materials

(Hamza Mohamed)

	Introduction - [JAVA] #00	1
:		
	Installing the Java Development Tools (JDK, Apache NetBeans IDE) - [JAVA] #01	
:		2
	Escape Sequences and Comments - [JAVA] #02	
:		3
	Data Types - [JAVA] #03	
:		4
	Arithmetic Operators - [JAVA] #04	
:		5

Learn JAVA Programming From Scratch In Arabic

سلسلة دروس • Adel Nasim

77 فيديو تم إجراء التعديل الأخير في 2023/06/24

⋮ ⬇️ ↶️ 📖

التعليقات استئناف ▶️

[illegible]

Project: Library System

Description

The Library System is a Java-based console application designed to manage a collection of books in a library. It provides users with a menu-driven interface to perform essential operations such as adding new books, removing existing books, editing book information, searching for books by various criteria, and displaying the complete collection. This system demonstrates fundamental object-oriented programming concepts and serves as a practical solution for small-scale library management.

Key Components

Book Class: Represents a single book in the library, storing its title, author, and unique ISBN identifier. Provides getter and setter methods to access and modify book information, and a toString() method to display book details in a formatted way.

Library Class: Manages the collection of books using an ArrayList data structure. Handles all core operations including adding new books, removing

existing books by ISBN, and searching for books by title, author, or ISBN. Provides methods to display the complete book collection.

Search System: Implements three different search methods, title and author searches are case-insensitive with partial matching, while ISBN search requires an exact match. Search results are displayed in a clean, numbered format.

Interactive Menu System: A while loop with a switch statement in the Main class provides a user-friendly, menu-driven interface. Users can perform multiple operations in one session until they choose to exit. The menu displays six options: Add, Remove, Edit, Search, Display All, and Exit.

Input Validation: Includes safety features to prevent program crashes and invalid operations.

Summary of Operations

Action	Description
Add a Book	Creates a new Book object and adds it to the library collection.
Remove a Book	Searches for the book by ISBN and removes it from the collection if found.

Edit a Book	Displays current values in brackets and allows user to enter new values or press Enter to keep existing ones.
Search for Books	Presents a sub-menu with three search options: by Title, by Author, or by ISBN. Performs case-insensitive partial matching for title and author searches, and exact matching for ISBN.
Display All Books	Shows the complete library collection with numbered listing. Displays each book's title, author, and ISBN in a formatted way.
Exit	Closes the Scanner resource, displays a thank you message, and terminates the program.

Screenshots of the App:

```

1  package librarysystem.model;
2
3  public class Book {
4      private String title;
5      private String author;
6      private String isbn;
7
8      // Constructor
9      public Book(String title, String author, String isbn) {
10         this.title = title;
11         this.author = author;
12         this.isbn = isbn;
13     }
14
15     // Getters and Setters
16     public String getTitle() {
17         return title;
18     }
19
20     public void setTitle(String title) {
21         this.title = title;
22     }
23
24     public String getAuthor() {
25         return author;
26     }
27
28     public void setAuthor(String author) {
29         this.author = author;
30     }
31
32     public String getIsbn() {
33         return isbn;
34     }
35 }

```

```

package librarysystem;

import librarysystem.model.Book;
import librarysystem.service.Library;
import java.util.List;
import java.util.Scanner;

public class LibrarySystem {
    private static Scanner scanner = new Scanner(System.in);
    private static Library library;

    public static void main(String[] args) {
        System.out.println("=== Welcome to the Library Management System ===");
        System.out.print("Enter library name: ");
        String libName = scanner.nextLine();

        library = new Library(libName);

        addSampleBooks();

        boolean running = true;

        while (running) {
            displayMenu();
            int choice = getUserChoice();

            switch (choice) {
                case 1:
                    addBook();
                    break;
                case 2:
                    removeBook();
                    break;
                case 3:
                    editBook();
                    break;
            }
        }
    }
}

```

Activate
Go to Settings

```

1 package librarysystem.service;
2
3 import librarysystem.model.Book;
4 import java.util.ArrayList;
5 import java.util.List;
6 import java.util.stream.Collectors;
7
8 public class Library {
9     private String name;
10    private ArrayList<Book> books;
11
12    public Library(String name) {
13        this.name = name;
14        this.books = new ArrayList<>();
15    }
16
17    public void addBook(Book book) {
18        books.add(book);
19        System.out.println("Added: " + book.getTitle());
20    }
21
22    public boolean removeBook(String isbn) {
23        for (int i = 0; i < books.size(); i++) {
24            if (books.get(i).getIsbn().equals(isbn)) {
25                Book removed = books.remove(i);
26                System.out.println("Removed: " + removed.getTitle());
27                return true;
28            }
29        }
30        System.out.println("Book with ISBN " + isbn + " not found.");
31        return false;
32    }
33
34    public boolean editBook(String isbn, String newTitle, String newAuthor, String newIsbn) {
35        Book book = searchByIsbn(isbn);
36        if (book != null) {
37            if (!newTitle.isEmpty()) {

```

Activate Windows
Go to Settings to activate Windows


```

public List<Book> searchByTitle(String titleQuery) {
    return books.stream()
        .filter(book -> book.getTitle().toLowerCase()
            .contains(titleQuery.toLowerCase()))
        .collect(Collectors.toList());
}

public List<Book> searchByAuthor(String authorQuery) {
    return books.stream()
        .filter(book -> book.getAuthor().toLowerCase()
            .contains(authorQuery.toLowerCase()))
        .collect(Collectors.toList());
}

public Book searchByIsbn(String isbn) {
    for (Book book : books) {
        if (book.getIsbn().equals(isbn)) {
            return book;
        }
    }
    return null;
}

public void displayAllBooks() {
    if (books.isEmpty()) {
        System.out.println("\nThe library has no books.");
        return;
    }

    System.out.println("\n=== " + name + " - All Books (" + books.size() + ") ===");
    for (int i = 0; i < books.size(); i++) {
        System.out.println((i + 1) + ". " + books.get(i));
    }
}

```

Activat
Go to Set

```

private static void displayMenu() {
    System.out.println("\n=== " + library.getName() + " Menu ===");
    System.out.println("1. Add a book");
    System.out.println("2. Remove a book");
    System.out.println("3. Edit a book");
    System.out.println("4. Search for books");
    System.out.println("5. Display all books");
    System.out.println("6. Exit");
    System.out.print("Enter your choice (1-6): ");
}

private static int getUserChoice() {
    try {
        return Integer.parseInt(scanner.nextLine());
    } catch (NumberFormatException e) {
        return -1;
    }
}

private static void addSampleBooks() {
    library.addBook(new Book("Atomic Habits", "James Clear", "1-1-1-1"));
    library.addBook(new Book("The 7 Habits of Highly Effective People", "Stephen R. Covey", "1-1-1-2"));
    library.addBook(new Book("Fourty", "Ahmed Al-Shugairi", "1-1-1-3"));
    library.addBook(new Book("The Great Gatsby", "F. Scott Fitzgerald", "1-1-1-4"));
    System.out.println("\nSample books added to the library!");
}

private static void addBook() {
    System.out.println("\n--- Add a New Book ---");
    System.out.print("Enter title: ");
    String title = scanner.nextLine();
    System.out.print("Enter author: ");
    String author = scanner.nextLine();
    System.out.print("Enter ISBN: ");
    String isbn = scanner.nextLine();
}

```

Activate Windows
Go to Settings to activate Windi

```
run:
=== Welcome to the Library Management System ===
Enter library name: FCAI
Added: Atomic Habits
Added: The 7 Habits of Highly Effective People
Added: Fourty
Added: The Great Gatsby

Sample books added to the library!

=== FCAI Menu ===
1. Add a book
2. Remove a book
3. Edit a book
4. Search for books
5. Display all books
6. Exit
Enter your choice (1-6):
```

Video link:

<https://drive.google.com/file/d/1YOJGmYWFpn4endl77K8wvOeu5LP8cooh/view?usp=sharing>

Task 2

Zapier vs Bubble

First:

What is LCNC (Low-Code/No-Code)?

Low-Code/No-Code (LCNC) platforms are development tools that allow users to build software applications with minimal or no traditional programming. They use visual builders, drag-and-drop interfaces, prebuilt templates, and workflow logic to speed up development. LCNC tools help businesses rapidly create apps, automate processes, and prototype ideas while reducing development time and cost. Low-code platforms may allow some custom coding for flexibility, while no-code platforms are designed for users with no programming background.

What LCNC Platforms Can Do

Modern LCNC tools can:

- Build full web and mobile applications
- Automate business processes and workflows
- Create dashboards and internal management systems
- Develop e-commerce platforms and SaaS products
- Integrate multiple systems and databases
- Prototype ideas rapidly for testing and validation

Some low-code platforms also allow custom code extensions, offering greater flexibility for more advanced use cases.

Benefits

- Faster development: Projects can be completed in days or weeks instead of months.
- Lower costs: Reduces reliance on large development teams for smaller projects.
- Accessibility: Non-technical users (citizen developers) can build solutions.
- Rapid prototyping: Businesses can test ideas quickly before major investment.

- Improved agility: Companies can adapt workflows and systems quickly as needs change.

Quality of Systems Produced

The quality of LCNC-built systems depends on the platform and the complexity of the project. For small to medium-sized applications, internal tools, and early-stage startups, LCNC solutions can be highly professional, stable, and production-ready. However:

- Deep customization may be limited.
- Performance optimization can be constrained by the platform's infrastructure.
- Vendor lock-in can be a risk.
- Highly complex, large-scale, or performance-critical systems may require traditional development.

Overall, LCNC platforms are strong for standard business applications but less suited for highly specialized or technically demanding systems.

Will LCNC Replace Developers?

LCNC platforms are unlikely to eliminate the need for developers. Instead, they are reshaping the industry:

- They reduce the need for simple, repetitive coding tasks.
- They empower non-developers to solve basic business problems.
- They increase demand for developers in areas like system architecture, integration, security, scalability, and custom development.

Rather than replacing developers, LCNC tools complement traditional software development and shift professional developers toward more advanced, strategic, and technical work,

Now Onto the main show:

Bubble **vs** Zapier

Feature	Zapier	Bubbel
Primary Purpose	Workflow automation	Web app development
Main Function	Connects apps and automates tasks between them	Builds full web applications from scratch
Coding Required	No coding required	No coding required

Best For	Automating repetitive tasks	Building SaaS products, marketplaces, dashboards, etc.
User Interface	Simple trigger → action workflow builder	Visual drag-and-drop app builder
Database Support	No built-in database	Built-in database system
Customization Level	Limited to app integrations	High customization of UI and logic
Learning Curve	Very easy	Moderate (more features and complexity)
Example Use Case	When a new email arrives → save attachment to Google Drive	Build a job board website with user accounts

202404313.wixsite.com/za3bola

Welcome, FCI Com... Course | edX Source code - Virtu... CS112 Spring 2025... How to Use Cygwin... malware-traffic-anal... من الصفح إلى... I-Learn Anti CV - Online LaT... ONL3_DAT2_SS-F-to... »

Built on WIX Harmony

Za3bola Home Book Online Program List Notifications Blog Click Login

The task master for your school.

Za3bola streamlines task management between administrators



https://drive.google.com/file/d/1QJ62ls1OdRpNAukhxsRw5gjdTmN5u_Xf/view?usp=drive_link

Report: The FinTech Landscape in Egypt (2024-2025)

1. Introduction: The Concept of FinTech

Financial Technology, or **FinTech**, refers to the innovative use of technology to deliver financial services. It represents a shift from traditional, physical banking to digital-first solutions that prioritize speed, accessibility, and user experience. Key drivers for its global and local adoption include:

- **Smartphone Ubiquity:** The widespread use of mobile devices.
 - **Convenience:** The demand for instant, 24/7 financial management.
 - **Financial Inclusion:** Reaching populations that were previously ignored by traditional banks.
-

2. Egypt's Position as a Regional Leader

Egypt has rapidly evolved into a primary FinTech hub within the MENA region due to several strategic advantages:

A. Market Dynamics

- **Young Demographics:** With a median age of 24.2, Egypt possesses a tech-native population eager for digital solutions.
- **Internet Access:** High connectivity rates (over 72%) provide the necessary infrastructure for app-based services.

B. Regulatory and Growth Milestones

- **Government Strategy:** The Central Bank of Egypt (CBE) launched a dedicated FinTech strategy in 2019 to foster a robust ecosystem.
 - **Financial Inclusion:** The percentage of citizens with financial access jumped from 12% in 2014 to over 70% by 2023.
 - **Industry Recognition:** In 2024, Egypt had 13 companies on the Forbes Middle East FinTech 50 list, the highest number for any country in the region.
-

3. Major FinTech Segments and Market Leaders

The Egyptian market is diverse, ranging from simple payments to complex investment tools.

Category	Top Examples	Key Market Role
Digital Payments	Fawry, Paymob	Facilitating daily transactions and merchant services.
Super-Apps & Lending	MNT-Halan	Offering a mix of micro-loans and lifestyle services.
Financing (BNPL)	valU	Providing flexible installment plans for retail purchases.
Investment	Thndr	Simplifying stock market access for first-time investors.
Savings Circles	Money Fellos	Digitizing the traditional "Gameya" model for millions.

4. Ethical Classification: Halal vs. Impermissible FinTech

For Egyptian consumers, Sharia compliance is a critical factor in choosing a FinTech service. Below is a breakdown of the ethical status of common activities:

Activity	Ruling	Reasoning
Digital Wallets	Halal	Viewed as a neutral tool for storing and transferring existing cash.
Installments (BNPL)	Conditional	Halal if interest-free; if late fees are structured as interest, it becomes Haram.
Digital Lending	Generally Haram	Prohibited if the model is based on charging interest (Riba) on the loan.
Stock Investing	Conditional	Halal if the user invests only in companies with Sharia-compliant operations.
Cryptocurrency	Debated	Often avoided due to high volatility (Gharar) and lack of underlying value.

5. Challenges and Strategic Path Forward

While the growth is impressive, the sector faces specific "bottlenecks":

- **Currency Volatility:** Devaluation increases costs for tech imports and affects startup valuations.
- **Infrastructure Gaps:** Digital services are still heavily concentrated in Cairo and the Delta, leaving rural areas underserved.
- **The Talent Gap:** Local startups often lose skilled developers to international companies offering higher salaries.

Recommendations for Future Growth:

1. **Unified Regulation:** Streamlining the licensing process between the CBE and the Financial Regulatory Authority (FRA).

2. **Rural Outreach:** Investing in digital infrastructure specifically for Upper Egypt and Sinai.
3. **Youth Focus:** Developing financial literacy apps for the 22% of the population under age 15.